Poutine Routine

A secure to-do application with a poutine collection

Course	**MOD8 - Advanced Web Programming (CEWP MOD8 GRB, Fall 2026)**
Institution	Concordia University, Continuing Education
Instructor	Marcos Sanches
Milestone	1 - Documentation and Problem Solution
Team	Eliot Cupillard, Abigaïl Mialaret, Romain Denoual, Tiffany Vongchanh
Date	21 September 2026
Repository	https://github.com/Tezay/poutine-routine

Contents

Problem statement . . . 3
- Context . . . 3
- What goes wrong . . . 3
- The problem . . . 3
- Target user . . . 3
- Goals . . . 3
- Constraints . . . 3

Proposed solution . . . 4
- Overview . . . 4
- Core loop . . . 4
- Scope . . . 4
- Security approach . . . 4
- Architecture . . . 5

Entity-relationship diagram . . . 7
- Relationships . . . 8
- Tables . . . 8
- Three decisions . . . 8

Front-end mockups . . . 9
- Authentication . . . 9
- Tasks . . . 10
- Collection . . . 12
- States . . . 13

Project plan . . . 14
- Timeline . . . 14
- Risks . . . 15
- How we work . . . 15

Task distribution . . . 15
- Milestone 1 - who did what . . . 16

Tools . . . 16

Problem statement

Context

Our users are students. A student follows homework for several courses, a group project, and personal tasks. Each task is simple. Keeping the list up to date for fourteen weeks is not.

Every member of our team has used a to-do application before. Every member stopped after one or two weeks. These applications were fast and free. They gave no reason to come back.

What goes wrong

- **Finishing a task gives nothing back.** The task disappears and the screen gets emptier. The user sees no progress.
- **Due dates are passive.** The date only turns red after the deadline. This is too late.
- **Gamified applications hide the list.** They add levels, quests and several currencies. The game becomes the work.
- **Task data is personal.** A task list shows what a student works on and when they are away. Many student projects protect this data badly.

The problem

> Students stop using to-do applications because finishing a task feels like nothing. The gamified ones make the task list harder to use. We want a task manager that stays simple and fast, gives a small reward for each finished task, and protects user data.

Target user

One student, on a laptop, with ten to forty open tasks. This student opens the application at least once a day. There is no team: sharing is out of scope.

Goals

1. The user comes back the next day.
2. Creating and finishing a task is as fast as in a plain to-do list.
3. Every reward rule is clear without explanation.
4. The user sees a deadline before it is missed.
5. A user can read and write only their own data.

Constraints

The project is also a course deliverable:

- front end in **Vue.js**, back end in **Node.js**, database in **MySQL**;
- three task statuses: **To Do**, **Doing**, **Done**;
- authentication with **token-based session management**;
- three graded milestones, so the application must work at each stage;

- four students working in one repository for ten weeks.

Proposed solution

Overview

Poutine Routine is a to-do application with a small collection game.

The task side is a normal task manager: a kanban board with three columns, tags, due dates, priorities, filters, and a table view. The reward side is short: a finished task gives **fries**, fries buy a **box**, and a box gives a random **poutine**. A poutine is a pixel-art character with a name and a rarity. The user picks one poutine as a companion. It stays on screen and reacts to the tasks.

Three rules keep the product simple:

- the task list works even if the user ignores the game;
- there is one currency and one thing to buy with it;
- one image is enough to add a poutine, extra images come later.

Core loop

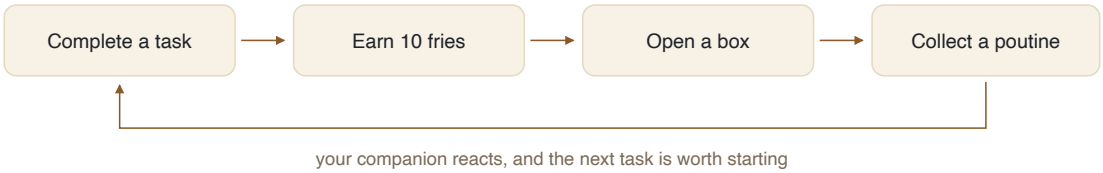


Figure 1: The core loop. One currency, four steps.

Scope

Priority	Features
Must have	Register, log in, log out, protected session. Task CRUD with title, description, status, due date, priority, tags. Kanban board with drag and drop and a keyboard equivalent. Tag filter. Fries on completion. Boxes and collection. Countdown and overdue alert.
Should have	Table view. Daily goal and streak. Companion with moods. Per-poutine effects. Dark theme.
Could have	Box opening animation, statistics, browser notifications, search.
Will not have	Sharing, multiple boards, real-time sync, password reset by email, third-party sign-in, mobile application.

Security approach

The application stores personal data. It also runs a small economy, and a user has a reason to cheat it. We handle both from the start.

Measure	Reason
Passwords hashed with bcrypt	A stolen database gives no usable password
Session JWT in an `httpOnly`, `SameSite` cookie, never in `localStorage`	JavaScript cannot read the token, so an XSS flaw cannot steal it
Secrets and database credentials in environment variables	A secret in the repository is public to the team and to anyone who clones it
Authentication middleware on the whole API	A new route is protected by default
Every query filtered by the user id from the token	A user cannot read or edit the tasks of another user
Server-side validation of every input	We do not trust the client
Same error message for unknown email and wrong password	Different messages tell an attacker which accounts exist
Rate limit on login and registration	Brute force becomes too slow to be useful
`helmet` headers, CORS limited to the client origin	Reduces browser-side attacks
The server computes the rewards, the client never sends an amount	The client cannot create fries
One SQL transaction to debit fries and give the poutine, with a condition on the balance	Two clicks at the same time cannot spend the same fries twice
Random draw on the server with a cryptographic generator	The client cannot choose its poutine
A task pays only once, stored in a timestamp	Unchecking and rechecking a task gives no new fries

Architecture

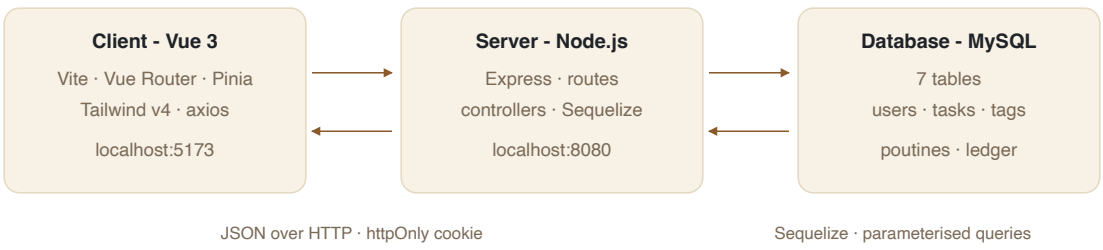


Figure 2: Three tiers. The client never talks to the database.

Layer	Choice	Reason
Front end	Vue 3 with Vite	Required by the course and used in the lectures
Routing	Vue Router 4	Needed to protect the pages
State	Pinia	Official state library for Vue 3
Styling	Tailwind CSS v4 with Primer colour tokens	Allowed by the brief, taught in class, and one file changes the whole theme
HTTP	axios, one shared instance with `withCredentials`	The session cookie must be sent with each request
Back end	Node.js with Express	Required by the course
ORM	Sequelize with `mysql2`	Taught in class. It gives parameterised queries and model validation
Database	MySQL 8, configured by each developer	Required by the course
Auth	bcryptjs, jsonwebtoken, cookie-parser	The stack shown in the course, plus the measures above

The repository holds `client/`, `server/` and `docs/`. Everything runs locally. We do not plan to deploy. The course teaches authentication in lesson 9, two weeks before the last milestone, so we will start it earlier with the course material that is already online.

Entity-relationship diagram

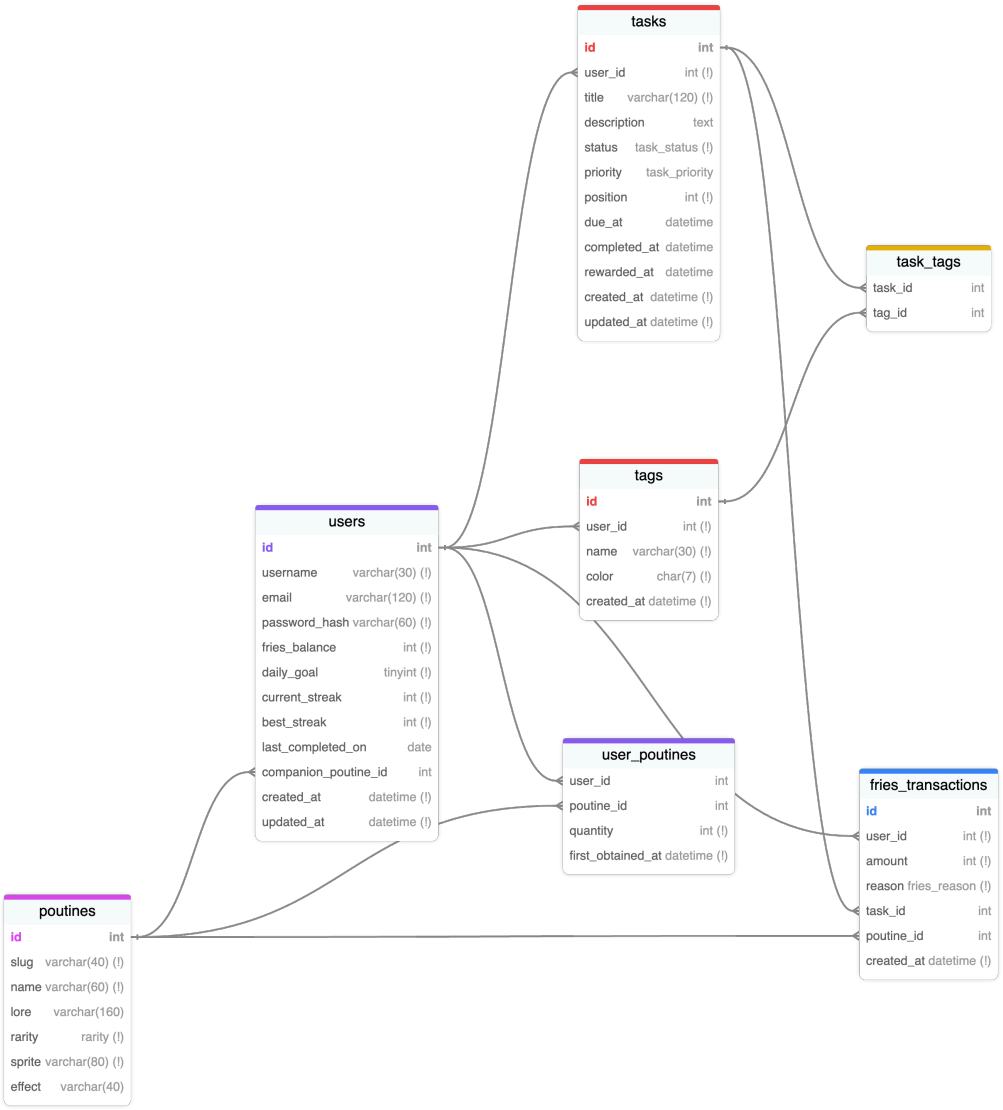


Figure 3: Seven tables. The source of this diagram is `docs/database/schema.dbml`, versioned with the code.

Relationships

Relationship	Type	Meaning
`users` → `tasks`	one to many	A task belongs to one user
`users` → `tags`	one to many	Tags belong to one user and are not shared
`tasks` ↔ `tags`	many to many, through `task_tags`	A task has several tags. A tag is used by several tasks
`users` ↔ `poutines`	many to many, through `user_poutines`	The join table also stores the number of copies and the first date
`users` → `fries_transactions`	one to many	Each change of the balance is one row
`users` → `poutines`	optional one to one	`companion_poutine_id` is the poutine on screen. It can be null

Tables

- **`users`** - account and game state. The password is stored only as a bcrypt hash. `fries_balance` is a copy of the total and can be rebuilt from `fries_transactions`.
- **`tasks`** - title, description, status (`todo`, `doing`, `done`), priority, `position` for the order in a column, `due_at` for the countdown, `completed_at` and `rewarded_at`.
- **`tags`** - name and colour, unique per user. Tags are the only way to sort the board.
- **`task_tags`** - join table. Its primary key is the pair of foreign keys.
- **`poutines`** - the list of characters. It is the same for every user and loaded at install.
- **`user_poutines`** - who owns what, and how many copies.
- **`fries_transactions`** - history of the currency: amount, reason, and the task or the poutine.

Three decisions

We keep a history, not only a balance. One `fries_balance` column would be enough. We also store each change as a row. A wrong total can then be explained and computed again.

`rewarded_at` is on the task. A user can mark a task done, undo it, and do it again to farm fries. We store the date of the payment, so the server pays only once.

No JSON column. Drop rates and image lists would be easy in JSON. Our machines run different local MySQL distributions and their JSON support is different. Drop rates stay in the server configuration. Image lists stay with the sprites in the client.

Front-end mockups

We first wrote the screens in HTML and CSS. We then imported them into Figma, where they are editable and linked as a clickable prototype. The colours are the real values of the future client.

The colours and the controls come from GitHub Primer, including the board colours: green for To Do, amber for Doing, purple for Done. The layout comes from Notion: a left sidebar, a wide content area, few borders. The collection uses pixel art and a pixel font. The work area stays sober, the reward area does not.

Colours are design tokens in two layers, structure and palette. One file changes the whole theme. Four palettes exist today. Tailwind v4 will use the same token names.

Authentication

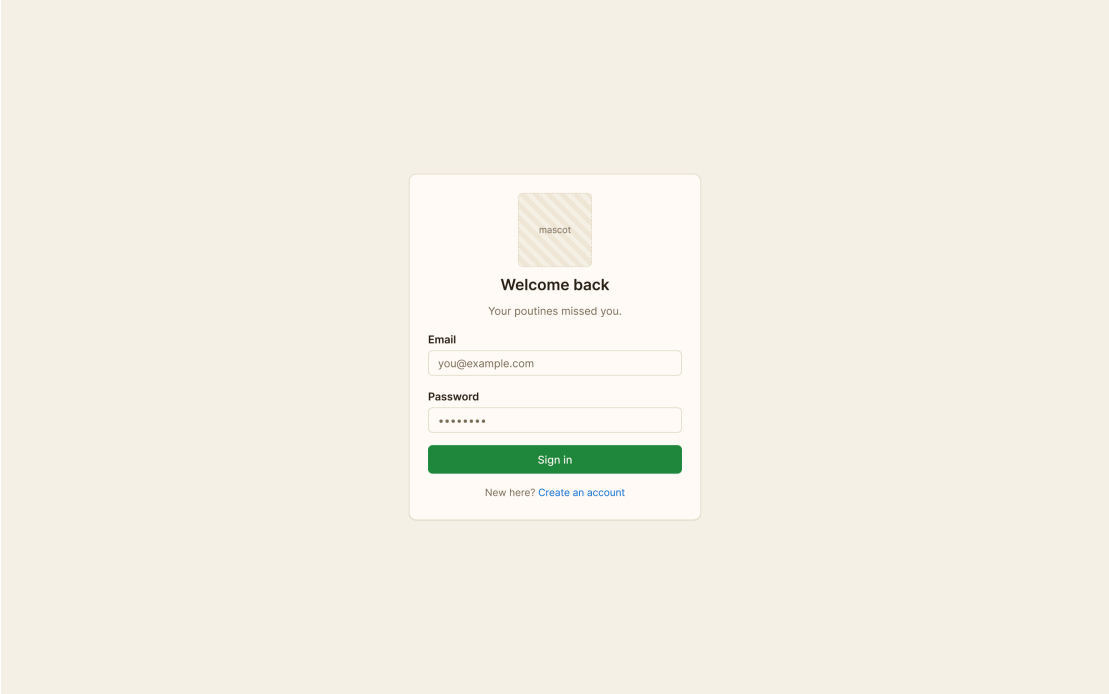


Figure 4: 01 - Login.

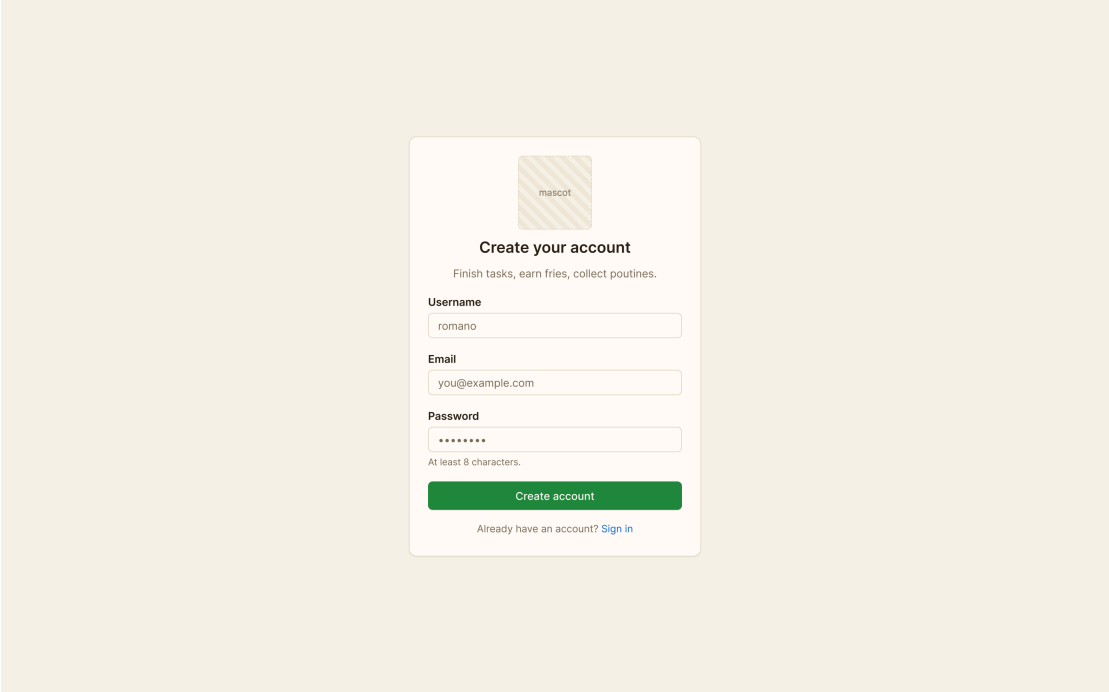


Figure 5: 02 - Register.

Tasks

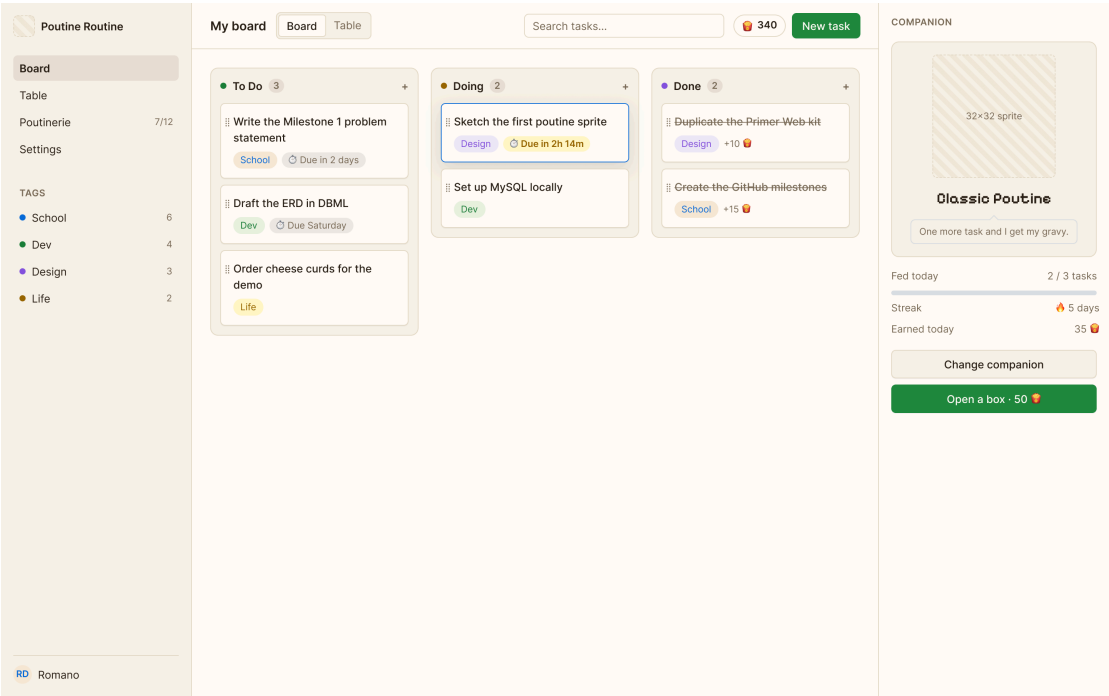


Figure 6: 03 - Board. Three columns, cards with tags, countdown and reward. Navigation on the left, companion on the right.

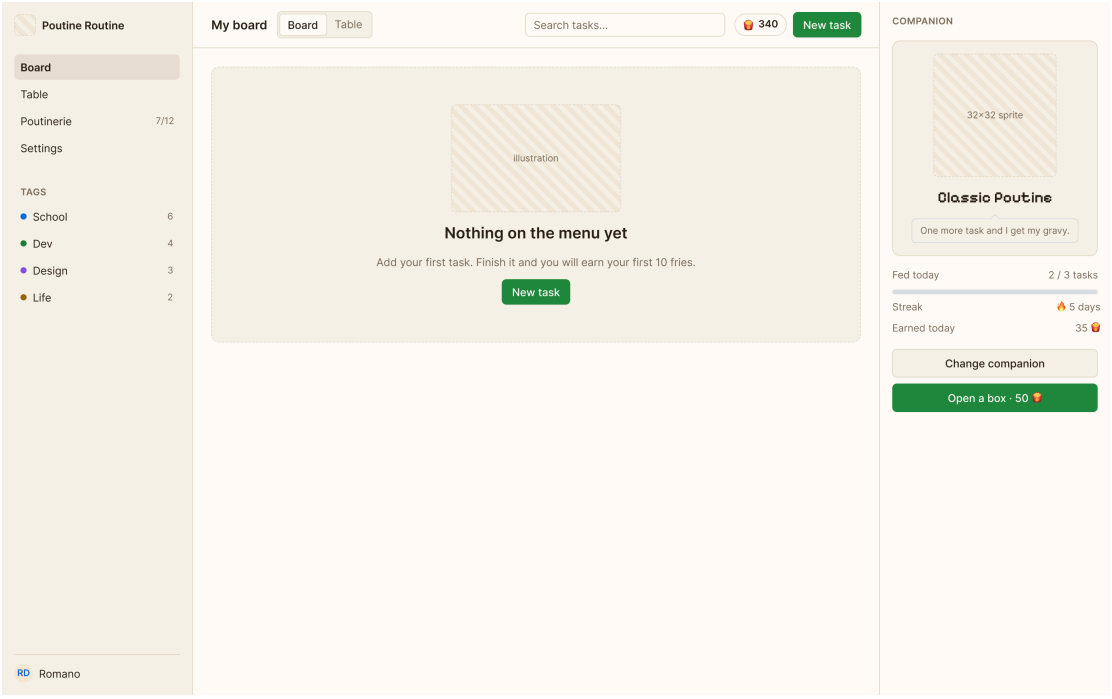


Figure 7: 04 - Empty board. First screen of a new account.

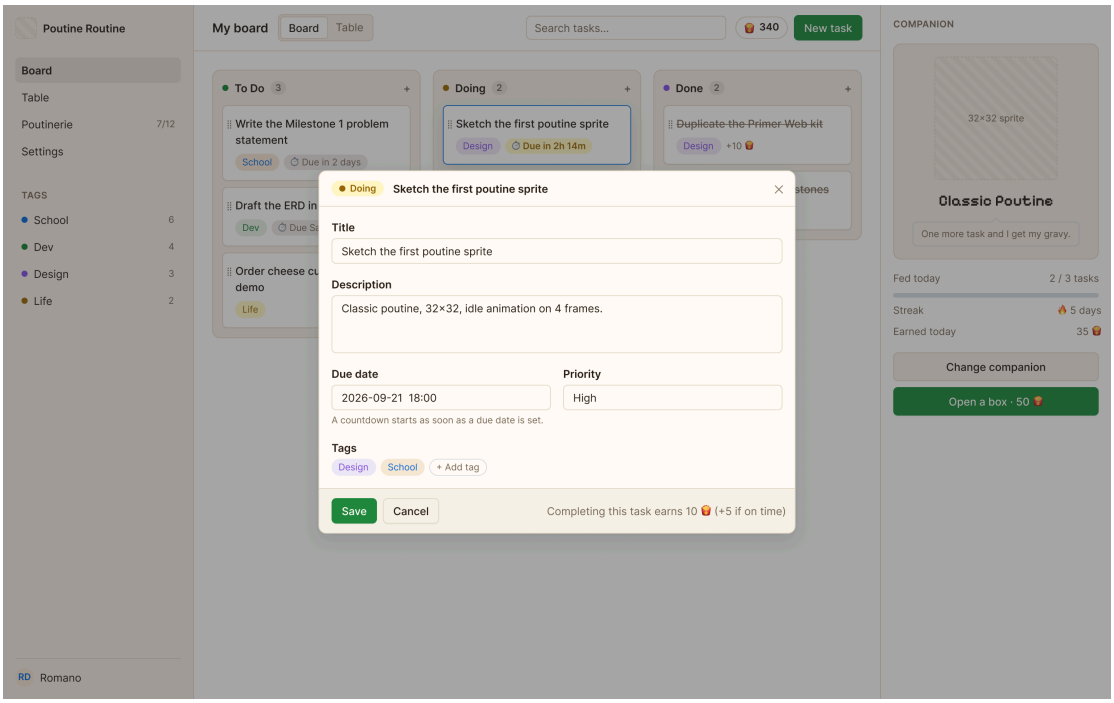


Figure 8: 05 - Task detail. Title, description, due date, priority and tags.

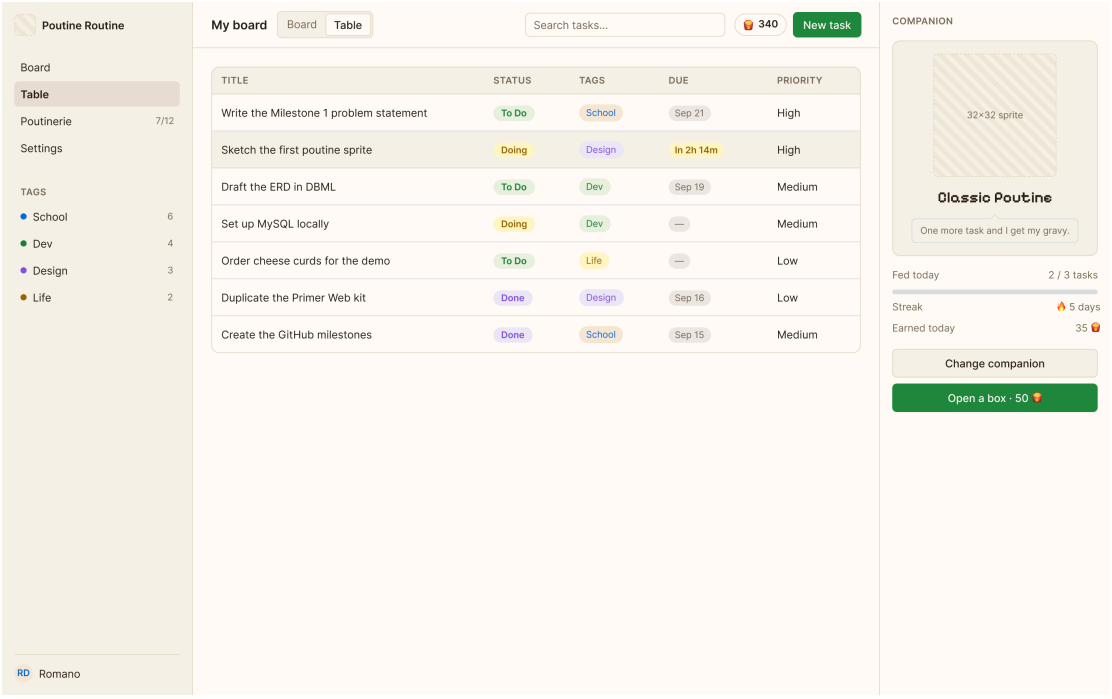


Figure 9: 06 - Table view, to sort and scan the tasks.

Collection

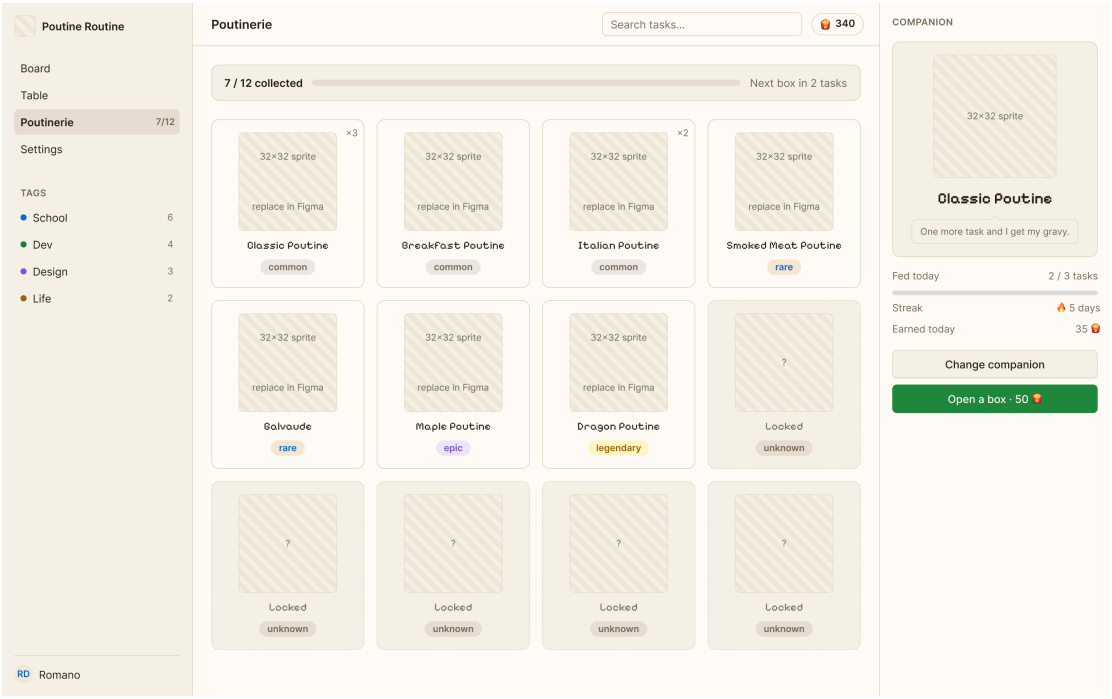


Figure 10: 07 - Poutinerie. Owned poutines with their rarity. Locked ones stay as silhouettes.

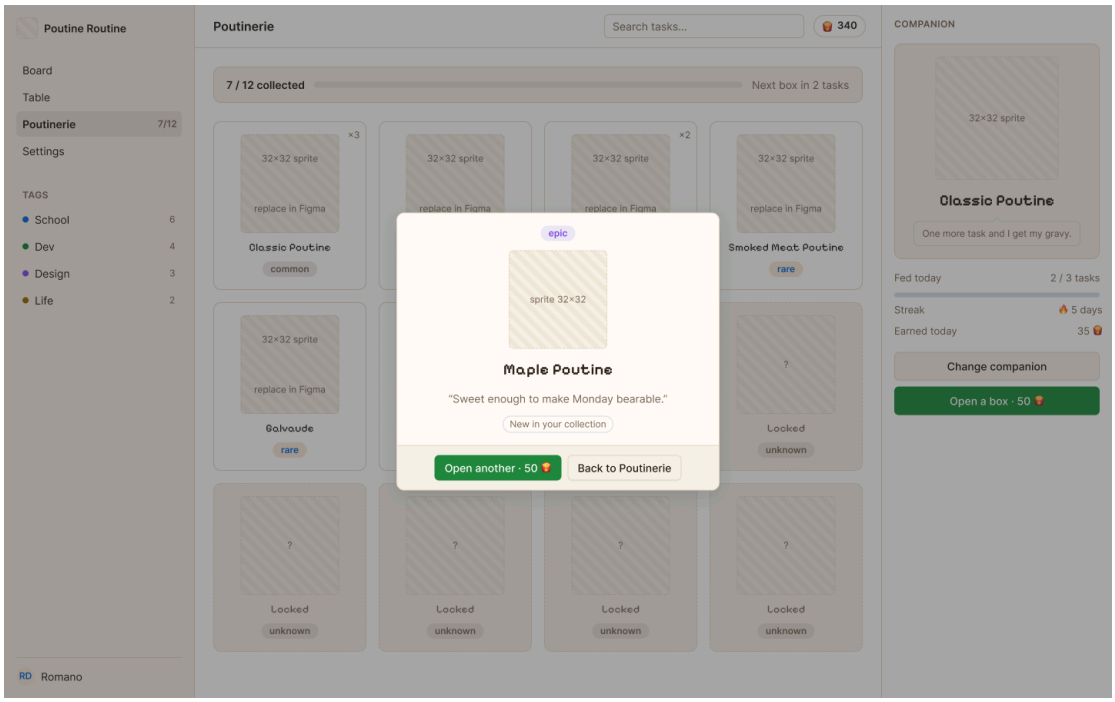


Figure 11: 08 - Opening a box.

States

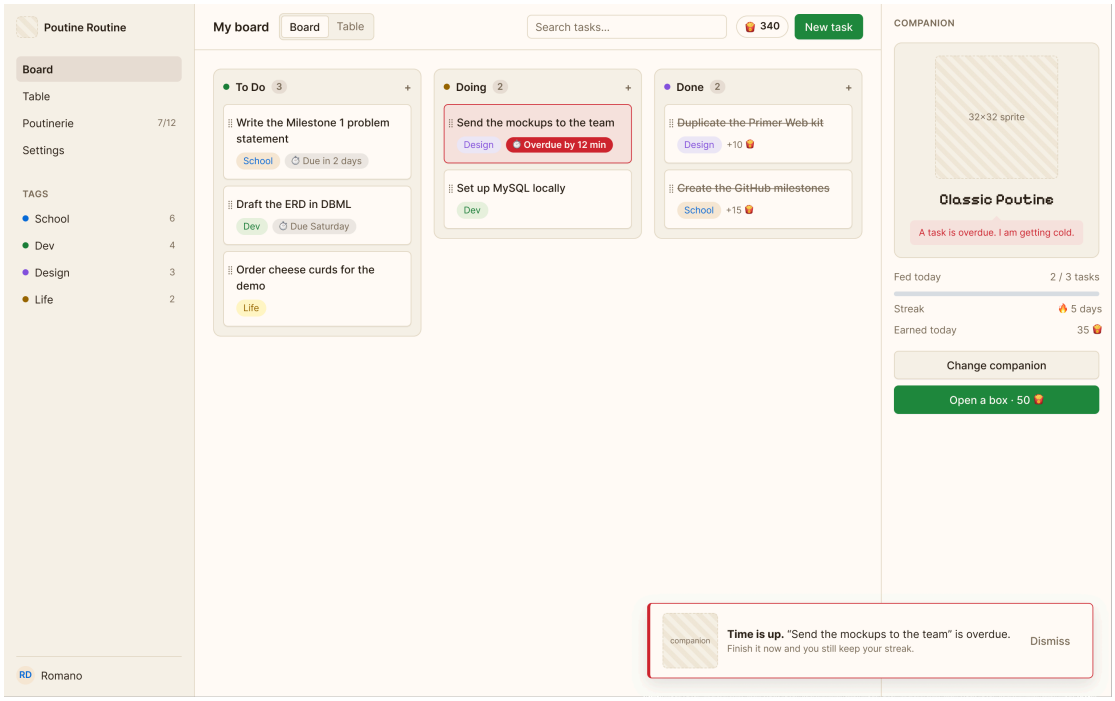


Figure 12: 09 - Overdue. The screen edge pulses red twice and the card turns red. Nothing moves if the system asks for reduced motion (WCAG 2.3.1).

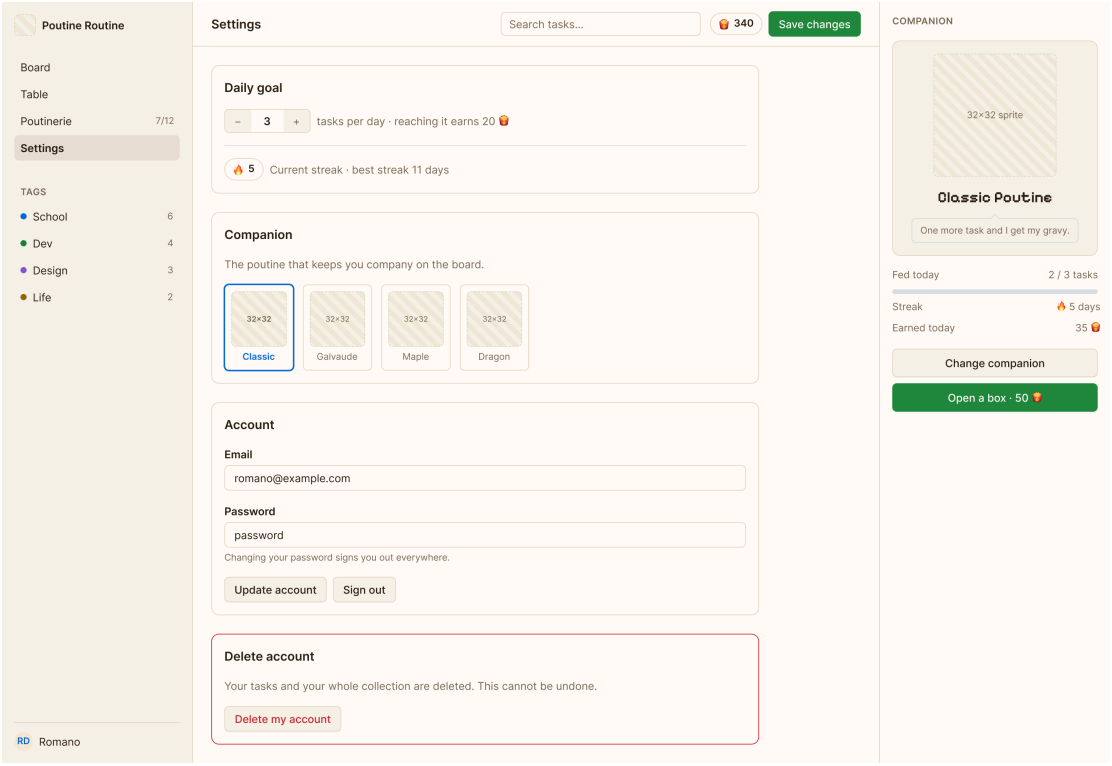


Figure 13: 10 - Settings. Daily goal, streak, companion, account.

Project plan

Timeline

The plan follows the milestone deadlines and the order of the lectures. The front end is taught first and is due first. The back end comes after.

Period	Sprint	Goal	Deliverable
8 - 21 Sep	0	Concept, scope, data model, mockups, plan	Milestone 1, 21 Sep
22 Sep - 4 Oct	1	Vite client, Express server, local MySQL, design tokens, application shell, authentication screens, board with mock data, first sprites	Running client
5 - 18 Oct	2	Full front end on mock data: drag and drop, tags, filters, countdown, table view, collection, box opening, companion	Milestone 2, 19 Oct
19 Oct - 1 Nov	3	Sequelize models, tasks and tags API, authentication, client connected to the real API	End-to-end application
2 - 15 Nov	4	Economy API, security work, poutine catalogue, tests, documentation, rehearsal	Milestone 3, 16 Nov

Quiz 1 is on 29 September, the midterm on 20 October, Quiz 2 on 3 November. We plan less project work on those weeks.

Risks

Risk	What we do
Authentication is taught two weeks before the deadline	We write it in sprint 3 with the course material that is already online
The front end is due before the database is taught	The front end runs on mock data first, like the course project
Pixel art depends on one person	One image is enough per poutine. Animations and moods are optional
Four different local MySQL installations	Each developer has a `.env` file. The schema works on MySQL and MariaDB
The game grows and the to-do suffers	The scope table in section 2 is fixed. New ideas become issues labelled `could`
Uneven work between members	Each member takes one feature, front and back, so everyone writes both

How we work

- **Repository:** one repository, `Tezay/poutine-routine`, with `client/`, `server/` and `docs/`.
- **Branches:** one branch per issue, named `feat/12-kanban-board`, `fix/...`, `docs/...`.
- **Reviews:** a merge into `main` needs a pull request and one approval. A repository ruleset blocks everything else. Merges are squashed, so the pull request title follows Conventional Commits.
- **Issues:** we write the issues before we start. Each issue has a type, an area and a milestone. Members take the issues they want.
- **Language:** English everywhere - interface, code, commits, issues, documentation.
- **Meetings:** we work in the same room every day. Decisions that affect the others are written in the repository.

Task distribution

Each member takes one feature, from the interface to the database, and reviews the pull requests of the others. We update this table at each milestone.

Slice	Scope	Owner
Authentication and security	Register, log in, session cookie, route guards, authentication middleware	Romain Denoual
Tasks and board	Task CRUD, kanban, drag and drop, tags, filters, table view	Tiffany Vongchanh
Deadlines and motivation	Countdown, overdue alert, daily goal, streak	Eliot Cupillard
Collection	Poutine catalogue, boxes, collection screen, companion, pixel art	Abigaïl Mialaret
Cross-cutting	Repository, continuous integration, database setup, design tokens, documentation	Eliot Cupillard

Milestone 1 - who did what

Contribution	Member
Problem statement and solution	Eliot Cupillard, Tiffany Vongchanh
Entity-relationship diagram	Eliot Cupillard, Romain Denoual
Mockups and design system	Abigaïl Mialaret, Eliot Cupillard
Project plan and repository setup	Eliot Cupillard
Review of this document	All members

Tools

Artefact	Tool
Mockups	Figma, with the official Primer Web component library. The images in this document are exported from Figma.
Entity-relationship diagram	Written as DBML in `docs/database/schema.dbml`, rendered with the DBML ERD Visualizer extension for VS Code
Diagrams (core loop, architecture)	SVG files generated with the help of an AI assistant (Claude) from our description, then reviewed and committed as `docs/assets/*.svg`
This document	Markdown files in `docs/milestone-1/`, converted to PDF with pandoc and the Typst engine
Source control	Git and GitHub, with issues, milestones, pull requests and a branch ruleset